MODUL 09 · BAB III · DIVIDE AND CONQUER DAN RECURRENCE

Recurrence dan Metode Master

Menentukan notasi asimtotik running time algoritma rekursif berbentuk $T(n) = aT(n/b) + f(n)$ secara cepat dan sistematis.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan arti recurrence dan makna parameter a, b, dan $f(n)$ pada $T(n) = aT(n/b) + f(n)$.
- Menyebutkan tiga metode penyelesaian recurrence dan gagasan dasar masing-masing.
- Menerapkan tiga kasus Metode Master, termasuk memeriksa syarat ε dan syarat keteraturan pada kasus 3.
- Mengenali recurrence yang tidak dapat diselesaikan dengan Metode Master.

PRASYARAT

- Modul 06–07: notasi Θ, O, Ω, dan perbandingan secara polinomial.
- Modul 08: recurrence Merge Sort dan pohon rekursi.

BAB / MATERI ASAL

Materi 7. Recurrence Metode Master

POSISI DALAM URUTAN BELAJAR

Alat analisis untuk algoritma Divide and Conquer (Modul 08). Menutup Bab III sebelum masuk ke Dynamic Programming.

DAFTAR ISI

1. Running Time yang Mengandung Dirinya Sendiri 1
2. Recurrence 2
3. Metode Master 3
4. Contoh-Contoh dari Bahan Kuliah 4
5. Batasan Metode Master 6
6. Kesimpulan 7
- Kuis Pemahaman 8
- Kunci Jawaban dan Pembahasan 9
- Referensi 9

PENDAHULUAN

1. Running Time yang Mengandung Dirinya Sendiri

Di Modul 08 kita menurunkan running time Merge Sort sebagai $T(n) = 2T(n/2) + \Theta(n)$. Persamaan ini tidak langsung memberi tahu seberapa cepat Merge Sort, karena T muncul di kedua ruas. Kita sempat menyelesaikannya secara intuitif dengan pohon rekursi dan memperoleh $\Theta(n \lg n)$.

Cara itu bekerja, tetapi melelahkan bila harus diulang untuk setiap algoritma rekursif. Untungnya, sebagian besar algoritma Divide and Conquer menghasilkan recurrence dengan bentuk yang sama: a **submasalah, masing-masing berukuran** n/b**, ditambah biaya** $f(n)$ **untuk membagi dan menggabungkan**. Untuk bentuk ini ada "resep" siap pakai yang disebut **Metode Master**.

Modul ini membahas apa itu recurrence, sekilas tiga metode penyelesaiannya, lalu Metode Master secara mendalam: syaratnya, ketiga kasusnya, cara memeriksanya langkah demi langkah, dan batasan pemakaiannya.

PEMBAHASAN INTI

2. Recurrence

DEFINISI

Recurrence adalah persamaan atau pertidaksamaan yang menyatakan suatu fungsi dalam bentuk nilai fungsi itu sendiri untuk masukan yang lebih kecil. Dalam analisis algoritma, recurrence adalah fungsi running time yang di dalamnya mengandung fungsi running time.

Recurrence muncul secara alami dari algoritma rekursif. Bentuk umum yang dibahas di modul ini adalah

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n),$$

dengan arti setiap bagiannya:

Simbol	Arti	Syarat	Merge Sort	Binary Search
a	Banyaknya submasalah yang diselesaikan secara rekursif	$a \geq 1$	2	1
b	Faktor pembagi ukuran masalah (setiap submasalah berukuran n/b)	$b > 1$	2	2
$f(n)$	Waktu untuk membagi masalah dan menggabungkan hasil submasalah	positif untuk n besar	$\Theta(n)$	$\Theta(1)$

Tabel 1. Makna parameter recurrence Divide and Conquer.

DETAIL TEKNIS YANG DIABAIKAN

Jika n ganjil, Merge Sort sebenarnya menghasilkan submasalah berukuran $\lceil n/2 \rceil$ dan $\lfloor n/2 \rfloor$, bukan tepat $n/2$. Selain itu ada kasus dasar $T(1) = \Theta(1)$. Cormen dkk. (2009) menunjukkan bahwa pembulatan dan kasus dasar seperti ini tidak mengubah hasil asimtotik, sehingga dalam praktik keduanya boleh diabaikan.

2.1 Tiga metode penyelesaian recurrence

Bahan kuliah menyebutkan tiga metode untuk menentukan notasi asimtotik suatu recurrence:

1. **Metode substitusi**: tebak bentuk solusinya, lalu buktikan tebakan itu dengan induksi matematika. Metode ini paling umum, tetapi membutuhkan tebakan yang baik.
2. **Metode pohon rekursi** (*recursion-tree method*): gambarkan biaya setiap pemanggilan sebagai simpul pohon, jumlahkan biaya per tingkat, lalu jumlahkan semua tingkat. Metode ini sangat baik untuk membangun intuisi, seperti yang kita lakukan pada Merge Sort di Modul 08.

3. **Metode Master**: rumus siap pakai untuk recurrence berbentuk $T(n) = aT(n/b) + f(n)$. Metode inilah yang dibahas secara mendalam di modul ini.

PEMBAHASAN INTI

3. Metode Master

3.1 Gagasan: siapa yang mendominasi pohon rekursi?

Bayangkan pohon rekursi untuk $T(n) = aT(n/b) + f(n)$. Akar berbiaya $f(n)$. Tingkat berikutnya memiliki a simpul, masing-masing berbiaya $f(n/b)$. Pohon terus bercabang sampai ukuran submasalah menjadi 1, yaitu setelah $\log_b n$ tingkat. Banyaknya daun adalah

$$a^{\log_b n} = n^{\log_b a}.$$

(Kesamaan ini berlaku karena kedua ruas sama dengan $b^{\log_b a \cdot \log_b n}$.) Jadi total biaya daun adalah $\Theta(n^{\log_b a})$. Metode Master pada dasarnya membandingkan **biaya di akar**, yaitu $f(n)$, dengan **biaya di daun**, yaitu $n^{\log_b a}$:

- Jika daun mendominasi, total biaya ditentukan oleh daun: $\Theta(n^{\log_b a})$.
- Jika keduanya seimbang, setiap tingkat berbiaya kira-kira sama dan ada sekitar $\log_b n$ tingkat: $\Theta(n^{\log_b a} \lg n)$.
- Jika akar mendominasi, total biaya ditentukan oleh akar: $\Theta(f(n))$.

3.2 Pernyataan Metode Master

TEOREMA MASTER (CORMEN DKK., 2009, TEOREMA 4.1)

Misalkan $a \geq 1$ dan $b > 1$ konstanta, $f(n)$ suatu fungsi, dan $T(n) = aT(n/b) + f(n)$. Bandingkan $f(n)$ dengan $g(n) = n^{\log_b a}$:

1. **Kasus 1.** Jika $f(n) = O(n^{\log_b a - \varepsilon})$ untuk suatu konstanta $\varepsilon > 0$, maka $T(n) = \Theta(n^{\log_b a})$.
2. **Kasus 2.** Jika $f(n) = \Theta(n^{\log_b a})$, maka $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. **Kasus 3.** Jika $f(n) = \Omega(n^{\log_b a + \varepsilon})$ untuk suatu konstanta $\varepsilon > 0$, **dan** $af(n/b) \leq cf(n)$ untuk suatu konstanta $c < 1$ dan semua n yang cukup besar (**syarat keteraturan**), maka $T(n) = \Theta(f(n))$.

Beberapa hal penting:

- ε adalah bilangan positif (boleh kecil). Kasus 1 menuntut $f(n)$ lebih kecil dari $n^{\log_b a}$ **secara polinomial**, yaitu lebih kecil setidaknya dengan faktor n^{ε}. Kasus 3 menuntut sebaliknya.
- Untuk $f(n)$ berbentuk polinomial n^k, aturannya menjadi sederhana: bandingkan pangkat k dengan $\log_b a$. Jika $k < \log_b a$: kasus 1. Jika $k = \log_b a$: kasus 2. Jika $k > \log_b a$: kasus 3 (syarat keteraturan selalu terpenuhi untuk $f(n) = n^k$ dengan $k > \log_b a$, karena $a(n/b)^k = \frac{a}{b^k} n^k$ dan $\frac{a}{b^k} < 1$).

3.3 Prosedur langkah demi langkah

1. Pastikan recurrence berbentuk $T(n) = aT(n/b) + f(n)$ dengan $a \geq 1$, $b > 1$ konstanta. Tentukan a, b, dan $f(n)$. Jika ukuran submasalah ditulis seperti $\frac{3n}{5}$, ubah menjadi $\frac{n}{5/3}$ sehingga $b = \frac{5}{3}$.
2. Hitung $g(n) = n^{\log_b a}$.

3. Bandingkan $f(n)$ dengan $g(n)$ dan tentukan kasusnya. Untuk kasus 1 dan 3, sebutkan ε yang memenuhi.
4. Untuk kasus 3, periksa syarat keteraturan $af(n/b) \le cf(n)$ dengan $c < 1$.
5. Tuliskan solusinya sesuai kasus.

CONTOH DAN PENERAPAN

4. Contoh-Contoh dari Bahan Kuliah

CONTOH 1 (KASUS 1): $T(N) = 2T(N/2) + \sqrt{N}$

$a = 2$, $b = 2$, $f(n) = \sqrt{n} = n^{1/2}$. Memenuhi syarat Metode Master.

$g(n) = n^{\log_2 2} = n^1 = n$.

Pangkat n pada $f(n)$, yaitu $\frac{1}{2}$, lebih kecil dari pangkat pada $g(n)$, yaitu 1. Memang $f(n) = n^{1/2} = O(n^{1-\varepsilon})$ untuk $\varepsilon = \frac{1}{2}$ (atau ε positif yang lebih kecil). Ini **kasus 1**, sehingga

$$T(n) = \Theta(n^{\log_b a}) = \Theta(n).$$

Catatan kecil: bahan kuliah menulis $0 < \varepsilon < \frac{1}{2}$. Nilai $\varepsilon = \frac{1}{2}$ sendiri juga memenuhi, karena $n^{1/2} = O(n^{1/2})$. Yang penting cukup ada **satu** nilai $\varepsilon > 0$.

CONTOH 2 (KASUS 2): $T(N) = 4T(N/2) + N^2$

$a = 4$, $b = 2$, $f(n) = n^2$. $g(n) = n^{\log_2 4} = n^2$.

Pangkat pada $f(n)$ sama dengan pangkat pada $g(n)$, yaitu $f(n) = \Theta(g(n))$. Ini **kasus 2**:

$$T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(n^2 \lg n).$$

CONTOH 3 (KASUS 3): $T(N) = T(3N/5) + N$

Ubah bentuk $\frac{3n}{5} = \frac{n}{5/3}$, sehingga $a = 1$, $b = \frac{5}{3}$, $f(n) = n$. Memenuhi syarat ($a \ge 1$, $b > 1$).

$g(n) = n^{\log_{5/3} 1} = n^0 = 1$ (karena $\log_b 1 = 0$ untuk basis apa pun).

Pangkat pada $f(n)$, yaitu 1, lebih besar dari 0, dan $f(n) = n = \Omega(n^{0+\varepsilon})$ untuk $0 < \varepsilon \le 1$. Calon **kasus 3**. Periksa syarat keteraturan:

$$af\left(\frac{n}{b}\right) = 1 \cdot f\left(\frac{3n}{5}\right) = \frac{3}{5}n \le cn \quad \text{dengan } c = \frac{3}{5} < 1.$$

Syarat terpenuhi, sehingga $T(n) = \Theta(f(n)) = \Theta(n)$.

4.1 Soal latihan dari bahan kuliah

SOAL 1: $T(N) = 9T(N/3) + N$

$a = 9$, $b = 3$, $f(n) = n$. $g(n) = n^{\log_3 9} = n^2$.

Pangkat f (1) lebih kecil dari pangkat g (2), dan $f(n) = O(n^{2-\varepsilon})$ untuk $0 < \varepsilon \leq 1$. **Kasus 1**: $T(n) = \Theta(n^2)$.

SOAL 2: $T(N) = 5T(N/2) + N^3$

$a = 5$, $b = 2$, $f(n) = n^3$. $g(n) = n^{\log_2 5} \approx n^{2{,}32}$.

Pangkat f (3) lebih besar dari 2,32, dan $f(n) = \Omega(n^{2{,}32+\varepsilon})$ untuk $0 < \varepsilon \leq 0{,}68$. Calon **kasus 3**. Syarat keteraturan:

$$5f\left(\frac{n}{2}\right) = 5\left(\frac{n}{2}\right)^3 = \frac{5}{8}n^3 \leq c\,n^3 \quad \text{dengan } c = \frac{5}{8} < 1.$$

Jadi $T(n) = \Theta(f(n)) = \Theta(n^3)$.

4.2 Contoh tambahan

Recurrence	a, b	$n^{\log_b a}$	$f(n)$	Kasus	Solusi
$2T(n/2) + n$ (Merge Sort)	2, 2	n	n	2	$\Theta(n \lg n)$
$T(n/2) + 1$ (Binary Search)	1, 2	$n^0 = 1$	1	2	$\Theta(\lg n)$
$8T(n/2) + n^2$	8, 2	n^3	n^2	1 ($\varepsilon = 1$)	$\Theta(n^3)$
$7T(n/2) + n^2$	7, 2	$n^{\lg 7} \approx n^{2{,}81}$	n^2	1	$\Theta(n^{\lg 7})$
$3T(n/3) + n$	3, 3	n	n	2	$\Theta(n \lg n)$
$3T(n/4) + n \lg n$	3, 4	$n^{0{,}79}$	$n \lg n$	3 (lihat bawah)	$\Theta(n \lg n)$

Tabel 2. Penerapan Metode Master pada recurrence lain.

Baris terakhir adalah contoh dari Cormen dkk. (2009) dengan $f(n)$ yang bukan polinomial murni. Karena $n \lg n \geq n = n^{0{,}79+0{,}21}$ untuk $n \geq 2$, kita dapat memakai $\varepsilon \approx 0{,}2$. Syarat keteraturan: $3 \cdot \frac{n}{4} \lg \frac{n}{4} \leq \frac{3}{4} n \lg n$, sehingga $c = \frac{3}{4}$ memenuhi.

Recurrence $7T(n/2) + \Theta(n^2)$ adalah running time algoritma Strassen untuk perkalian matriks $n \times n$, yang lebih cepat dari cara biasa $\Theta(n^3)$ (Cormen dkk., 2009, Bab 4.2).

4.3 Memeriksa solusi secara numerik

Program berikut menghitung nilai recurrence secara langsung (dengan $T(1) = 1$ dan n berupa pangkat dari b), lalu membaginya dengan solusi dari Metode Master. Jika solusinya benar, perbandingannya mendekati suatu konstanta ketika n membesar.

Python: memeriksa solusi Metode Master

```python
import math
from functools import lru_cache

def recurrence(a, b, f):
    @lru_cache(maxsize=None)
    def T(n):
        return 1 if n <= 1 else a * T(n // b) + f(n)
    return T

lg = math.log2
P2 = [2**10, 2**15, 2**20]          # nilai n berupa pangkat 2
P3 = [3**6, 3**9, 3**12]            # nilai n berupa pangkat 3
uji = [  # (nama, a, b, f(n), solusi Metode Master, nilai n)
    ("2T(n/2)+n",   2, 2, lambda n: n,    lambda n: n * lg(n),   P2),
    ("4T(n/2)+n^2", 4, 2, lambda n: n*n,  lambda n: n*n * lg(n), P2),
    ("9T(n/3)+n",   9, 3, lambda n: n,    lambda n: n*n,         P3),
    ("5T(n/2)+n^3", 5, 2, lambda n: n**3, lambda n: n**3,        P2),
    ("T(n/2)+1",    1, 2, lambda n: 1,    lambda n: lg(n),       P2),
]
for nama, a, b, f, sol, ns in uji:
    T = recurrence(a, b, f)
    print(f"{nama:12s}", [round(T(n) / sol(n), 3) for n in ns])
```

Keluaran program

```
2T(n/2)+n    [1.1, 1.067, 1.05]
4T(n/2)+n^2  [1.1, 1.067, 1.05]
9T(n/3)+n    [1.499, 1.5, 1.5]
5T(n/2)+n^3  [2.652, 2.665, 2.667]
T(n/2)+1     [1.1, 1.067, 1.05]
```

Semua perbandingan mendekati konstanta (1; 1,5; atau $\frac{8}{3}$), sesuai dengan solusi Metode Master. Konstanta itu sendiri tidak penting; yang penting perbandingannya tidak terus membesar atau mengecil.

ANALISIS DAN HUBUNGAN KONSEP

5. Batasan Metode Master

Metode Master tidak mencakup semua recurrence. Ada tiga situasi umum ketika metode ini **tidak dapat** dipakai:

1. **Bentuknya tidak sesuai**, misalnya a atau b bukan konstanta, $a < 1$, atau ukuran submasalah berkurang secara pengurangan, seperti $T(n) = T(n-1) + n$ (running time rekursif yang mirip Insertion Sort). Recurrence itu harus diselesaikan dengan cara lain; hasilnya $\Theta(n^2)$.
2. **Celah antara kasus 1 dan 2, atau antara kasus 2 dan 3.** Contoh klasik (Cormen dkk., 2009): $T(n) = 2T(n/2) + n \lg n$. Di sini $g(n) = n$, dan $f(n) = n \lg n$ memang lebih besar dari n, tetapi **tidak secara polinomial**: perbandingannya $\frac{n \lg n}{n} = \lg n$, yang tumbuh lebih lambat dari n^{ε} untuk setiap $\varepsilon > 0$ (Modul 07). Jadi bukan kasus 2 maupun kasus 3. (Dengan metode lain diperoleh $T(n) = \Theta(n \lg^2 n)$.)
3. **Syarat keteraturan kasus 3 gagal.** Hal ini jarang terjadi untuk fungsi yang biasa muncul dalam analisis algoritma, tetapi tetap harus diperiksa.

Hubungan dengan modul lain. Metode Master menyelesaikan running time algoritma Divide and Conquer (Modul 08) dan Binary Search (Modul 02). Algoritma Dynamic Programming (Modul 10–12) biasanya dianalisis dengan cara berbeda, yaitu menghitung banyaknya sel tabel dikali biaya per sel, karena solusi submasalahnya disimpan dan tidak dihitung ulang. Asal-usul Metode Master dapat ditelusuri pada karya Bentley, Haken, dan Saxe (1980).

KESIMPULAN

6. Kesimpulan

- Recurrence adalah fungsi running time yang memuat dirinya sendiri; bentuk $T(n) = aT(n/b) + f(n)$ muncul dari algoritma Divide and Conquer dengan a submasalah berukuran n/b dan biaya bagi-gabung $f(n)$.
- Tiga metode penyelesaian: substitusi, pohon rekursi, dan Metode Master.
- Metode Master membandingkan $f(n)$ dengan $n^{\log_b a}$: lebih kecil secara polinomial → $\Theta(n^{\log_b a})$; sama → $\Theta(n^{\log_b a} \lg n)$; lebih besar secara polinomial dan memenuhi syarat keteraturan → $\Theta(f(n))$.
- Untuk $f(n) = n^k$, cukup bandingkan k dengan $\log_b a$.
- Metode Master tidak berlaku untuk recurrence yang bentuknya berbeda atau yang jatuh di celah antarkasus, seperti $2T(n/2) + n \lg n$.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 **PERHITUNGAN**

Tentukan kasus dan solusi Metode Master untuk $T(n) = 16T(n/4) + n$.

2 **PERHITUNGAN**

Tentukan kasus dan solusi untuk $T(n) = 2T(n/4) + \sqrt{n}$.

3 **PERHITUNGAN**

Tentukan kasus dan solusi untuk $T(n) = 3T(n/2) + n^2$, termasuk pemeriksaan syarat keteraturan.

4 **PILIHAN GANDA**

Pada recurrence $T(n) = aT(n/b) + f(n)$ untuk sebuah algoritma Divide and Conquer, $f(n)$ menyatakan ...

A. Banyaknya submasalah.
B. Ukuran setiap submasalah.
C. Waktu untuk membagi masalah dan menggabungkan hasil submasalah.
D. Running time kasus dasar.

5 **BENAR/SALAH DENGAN ALASAN**

"Recurrence $T(n) = 2T(n/2) + n \lg n$ dapat diselesaikan dengan kasus 3 Metode Master karena $n \lg n$ tumbuh lebih cepat dari n." Benar atau salah?

6 **ANALISIS KASUS**

Sebuah algoritma memecah masalah berukuran n menjadi 4 submasalah berukuran $n/2$ dan menggabungkannya dalam waktu $\Theta(n)$. Algoritma lain memecah menjadi 3 submasalah berukuran $n/2$ dengan biaya gabung yang sama. Hitung running time keduanya. Mana yang lebih efisien?

7 **PERHITUNGAN**

Tentukan solusi $T(n) = T(n/2) + n$. Algoritma macam apa yang mungkin menghasilkan recurrence ini?

8 **ANALISIS**

Mengapa Metode Master tidak dapat dipakai untuk $T(n) = T(n-1) + n$? Berapakah solusinya?

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

$a = 16$, $b = 4$, $g(n) = n^{\log_4 16} = n^2$, $f(n) = n = O(n^{2-\varepsilon})$ dengan $\varepsilon = 1$. **Kasus 1**: $T(n) = \Theta(n^2)$.

Soal 2

$a = 2$, $b = 4$, $g(n) = n^{\log_4 2} = n^{1/2}$, dan $f(n) = n^{1/2}$. Pangkatnya sama: **kasus 2**, sehingga $T(n) = \Theta(\sqrt{n} \lg n)$.

Soal 3

$a = 3$, $b = 2$, $g(n) = n^{\lg 3} \approx n^{1{,}585}$, $f(n) = n^2 = \Omega(n^{1{,}585+\varepsilon})$ untuk $0 < \varepsilon \leq 0{,}415$. Syarat keteraturan: $3\left(\frac{n}{2}\right)^2 = \frac{3}{4}n^2 \leq c\,n^2$ dengan $c = \frac{3}{4} < 1$. **Kasus 3**: $T(n) = \Theta(n^2)$.

Soal 4

Jawaban: C. Banyaknya submasalah adalah a dan ukurannya n/b. Running time kasus dasar biasanya dianggap $\Theta(1)$ dan tidak ditulis di $f(n)$.

Soal 5

Salah. Kasus 3 menuntut $f(n)$ lebih besar dari $n^{\log_b a} = n$ **secara polinomial**, yaitu $f(n) = \Omega(n^{1+\varepsilon})$ untuk suatu $\varepsilon > 0$. Perbandingan $\frac{n \lg n}{n} = \lg n$ tumbuh lebih lambat dari n^ε untuk setiap $\varepsilon > 0$, sehingga syarat itu tidak terpenuhi. Recurrence ini berada di celah antara kasus 2 dan 3.

Soal 6

Algoritma pertama: $a = 4$, $b = 2$, $g(n) = n^2$, $f(n) = n$, kasus 1, sehingga $\Theta(n^2)$. Algoritma kedua: $a = 3$, $b = 2$, $g(n) = n^{\lg 3} \approx n^{1{,}585}$, $f(n) = n$, kasus 1, sehingga $\Theta(n^{\lg 3}) \approx \Theta(n^{1{,}585})$. **Algoritma kedua lebih efisien**: mengurangi satu submasalah menurunkan eksponen dari 2 menjadi sekitar 1,585. (Ide ini dipakai pada algoritma perkalian bilangan bulat besar milik Karatsuba.)

Soal 7

$a = 1$, $b = 2$, $g(n) = n^0 = 1$, $f(n) = n = \Omega(n^{0+\varepsilon})$ dengan $\varepsilon = 1$. Keteraturan: $f(n/2) = \frac{n}{2} \leq \frac{1}{2}n$, sehingga $c = \frac{1}{2}$. **Kasus 3**: $T(n) = \Theta(n)$. Contohnya algoritma yang memeriksa seluruh data (biaya n), lalu melanjutkan secara rekursif hanya pada separuhnya, seperti varian algoritma seleksi/pencarian yang membuang separuh data pada setiap langkah.

Soal 8

Ukuran submasalahnya $n - 1$, bukan n/b untuk suatu konstanta $b > 1$, sehingga bentuknya tidak sesuai syarat Metode Master. Dengan menjabarkan: $T(n) = n + (n-1) + (n-2) + \cdots + 1 = \frac{n(n+1)}{2}$ (ditambah konstanta kasus dasar), sehingga $T(n) = \Theta(n^2)$.

Referensi

Sumber utama: Materi 7. Recurrence Metode Master (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Bentley, J. L., Haken, D., & Saxe, J. B. (1980). A General Method for Solving Divide-and-Conquer Recurrences. *ACM SIGACT News*, 12(3), 36–44. https://doi.org/10.1145/1008861.1008865